



Artificial Intelligence Foundations (CIS4049-N)

Problem-solving II: Uninformed Search – Part 2

Module Leader: Dr Alessandro Di Stefano

Email: A.DiStefano@tees.ac.uk

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk

Today's Plan

- **Searching & Reinforcement Learning**
- **Uninformed Search algorithms:**
 - **Iterative Deepening**
 - **Cost-Sensitive search**
 - **Uniform Cost search**

Searching & Reinforcement Learning

Search vs. RL

- Searching and RL are all techniques used in AI, but they serve different purposes and operate in distinct ways.
- **Searching:**
 - Searching algorithms are used to explore a problem space and find a solution. Uninformed methods don't use any domain-specific knowledge (systematic space exploration), whereas informed methods use heuristic functions to guide the search toward the goal (bias towards promising paths). However, they do not involve the concept of exploitation and do not learn from previous searches.
 - Searching algorithms receive feedback in the form of **success** or **failure** (finding a solution or not). They **do not receive explicit rewards or penalties** for their actions during the search process.
 - Searching algorithms operate in **static environments** where the problem space and the goal are well-defined and do not change over time.
 - Searching algorithms focus on finding a solution to a specific problem. They **do not learn or adapt based on their previous search experiences.**

Search vs. RL

- **Reinforcement Learning:**

- RL is a type of machine learning where an agent learns to make decisions by interacting with an environment. The agent takes actions in an environment and receives rewards as feedback. The goal is to **learn a policy that maps states to actions to maximize the cumulative reward.** RL algorithms learn from interaction with the environment. They don't require a priori knowledge about the environment; instead, they learn by trial and error. They improve their decision-making abilities over time through learning and experience.
- RL involves a balance between exploration and exploitation. The agent explores new actions to discover their rewards but also exploits known actions to maximize its reward based on past experiences. RL agents receive explicit **rewards** or **penalties** as feedback for each action taken. This feedback guides the agent's **learning process.**
- RL algorithms can operate in **dynamic environments** where the relationship between actions, states, and rewards might change over time. The agent needs to adapt its policy based on the changing environment.

In summary, searching methods are systematic techniques for finding solutions in a problem space, whereas RL is a dynamic learning process where an agent learns to make decisions to maximize rewards through interactions with an environment. Each approach is suitable for different types of problems and scenarios.

Iterative Deepening Depth-First Search

Uniform Cost Search

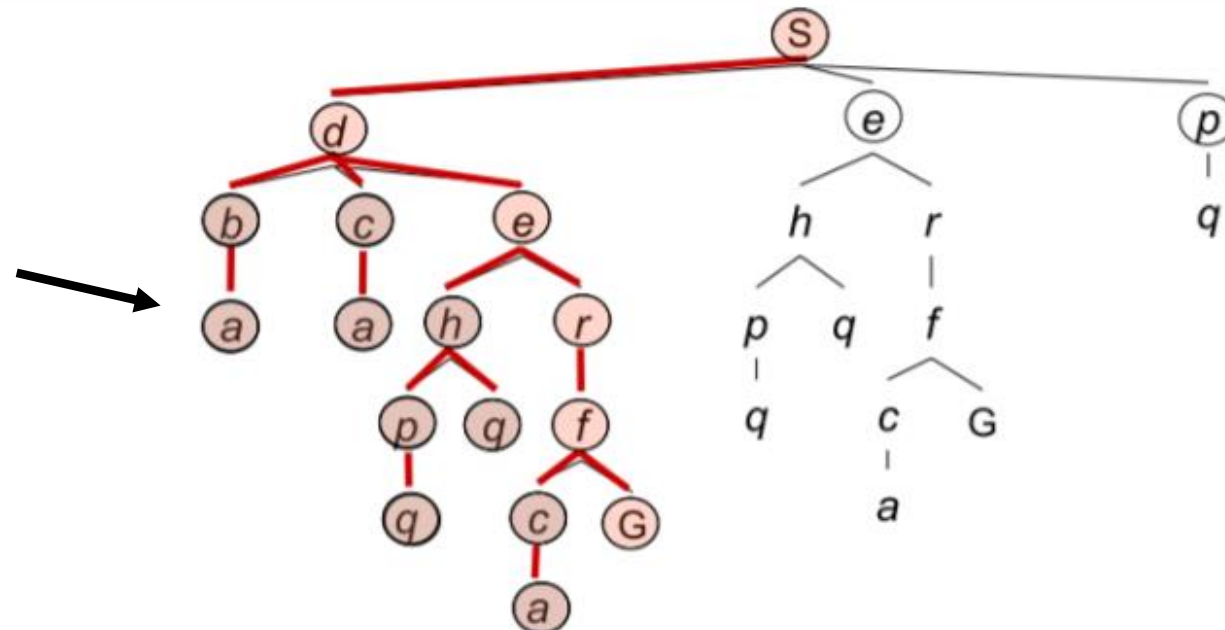
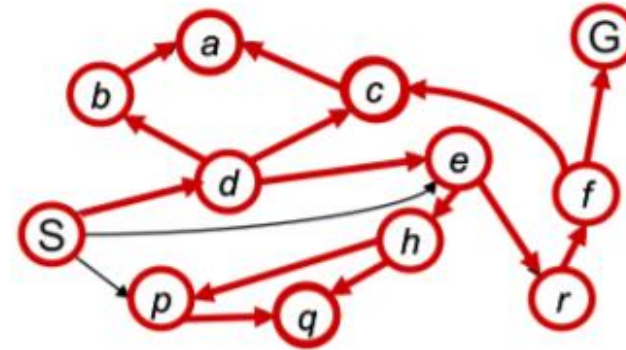
Depth-First Search (DFS)



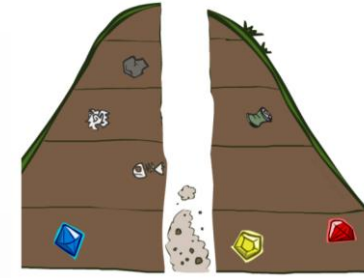
Tree Search - Example

*Strategy: expand a
deepest node first*

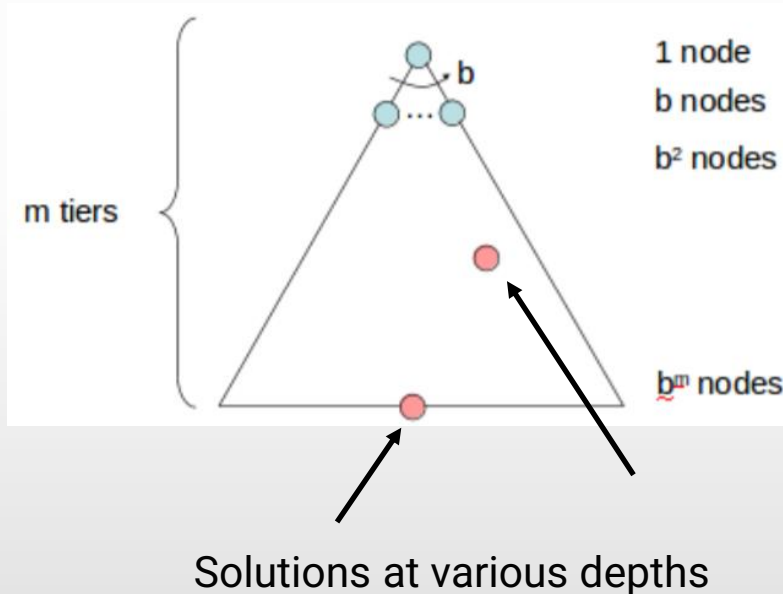
*Implementation:
Fringe is a LIFO stack*



Search Algorithms – Properties



Search tree cartoon

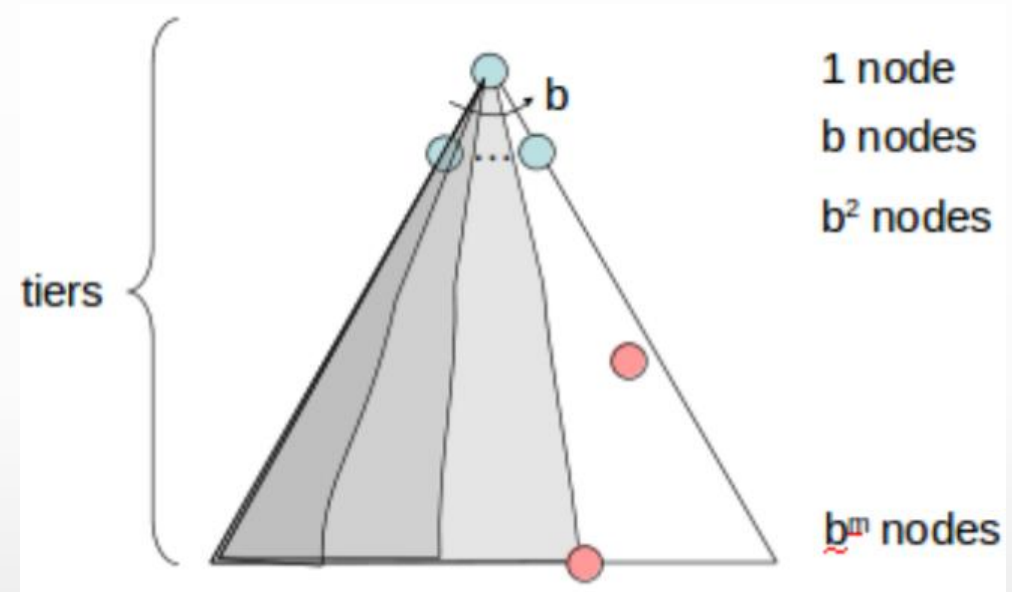


- **Complete:** Guaranteed to find a solution if one exists?
- **Optimal:** Guaranteed to find the least cost path?
- **Time complexity?** - how long does the compute take to find a solution?
- **Space complexity?** - how much memory do you need in the process of that compute to get the computation done?
- Sketch of search tree:
 - **b** is the **branching factor** (how many successors are there from any given node?)
 - **m** is the **maximum depth**
 - **solutions** at various depths
- Number of nodes in entire tree?

$$1 + b + b^2 + \dots = O(b^m)$$

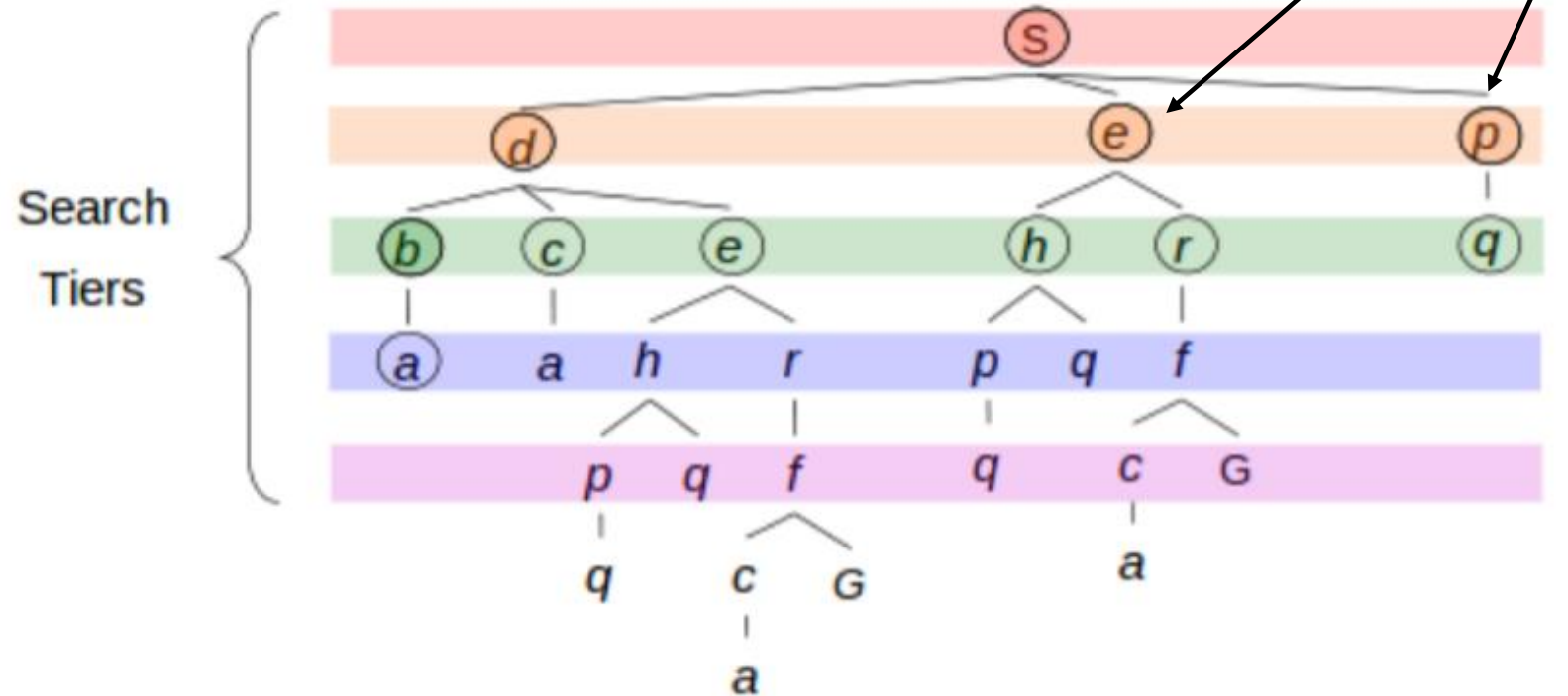
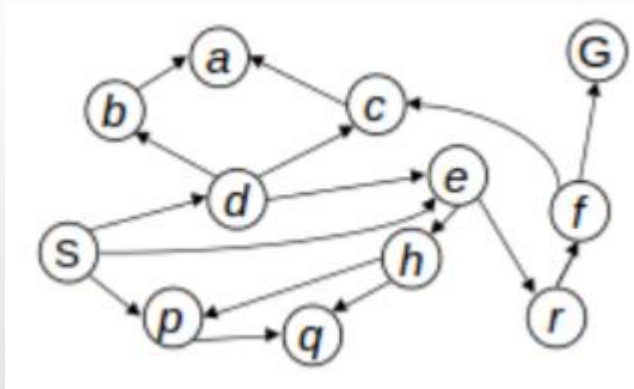
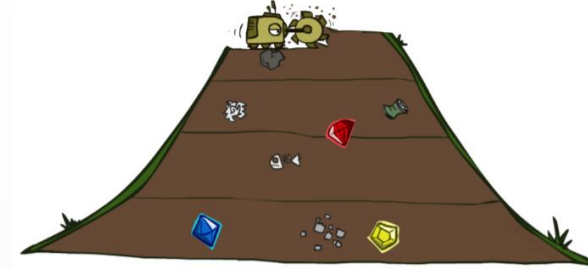
Depth-First Search – Properties

- Which nodes expanded?
 - Some left prefix of the tree
 - Could process the whole tree!
 - If m is finite, takes time $O(b^m)$
- How much space does the fringe take?
 - Only has siblings on path to root, so $O(b^m)$
- Is it complete?
 - m could be infinite, so only if we prevent cycles (more later)
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost



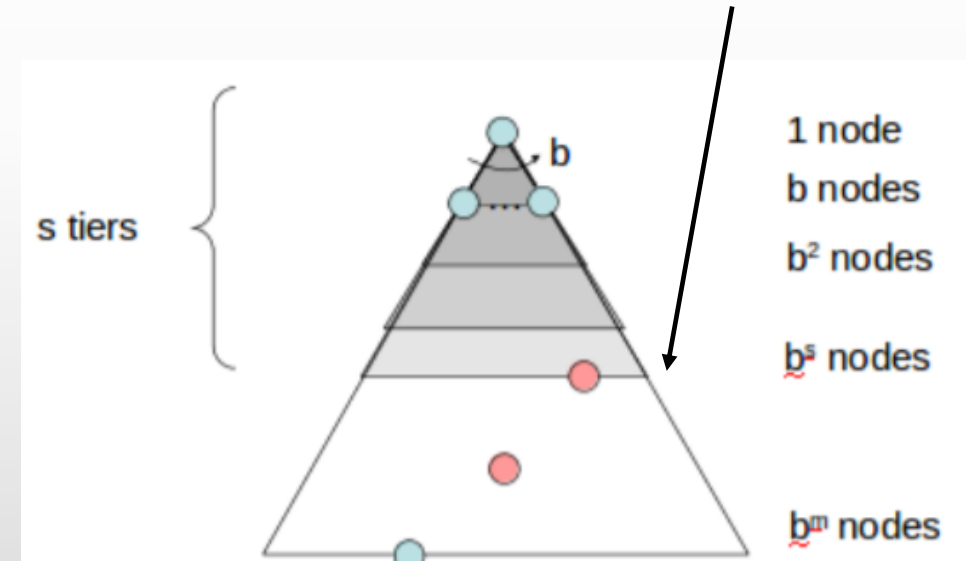
Breadth-First Search (BFS)

- **Strategy:** expand a shallowest node first
- **Implementation:** Fringe is a FIFO stack



Breadth-First Search - Properties

- What nodes does BFS expand?
 - Processes all nodes above shallowest solution
 - Let depth of shallowest solution be $s \rightarrow$ Search takes time $O(b^s)$
- How much space does the fringe take?
 - Has roughly the last tier, so $O(b^s)$
- Is it complete?
 - s must be finite if a solution exists, so yes!
- Is it optimal?
 - Only if costs are all 1 (more on costs later)

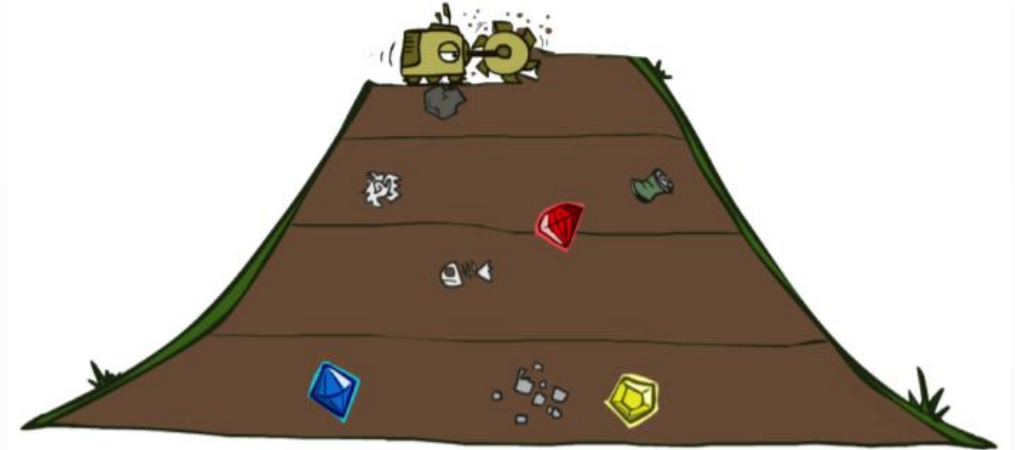


DFS vs BFS



DFS

VS

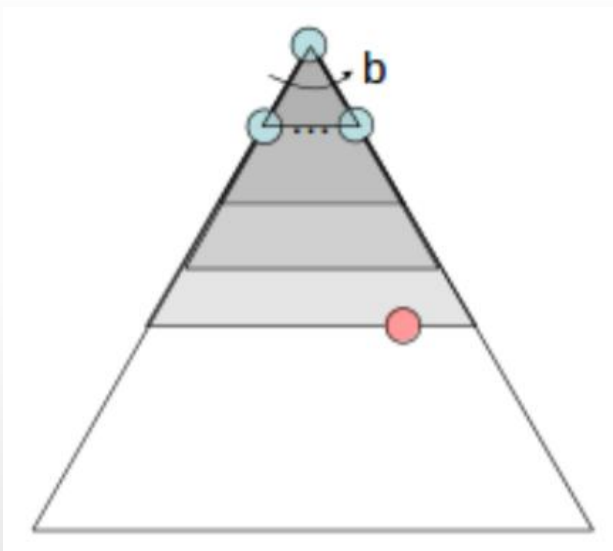


BFS

- When will BFS outperform DFS? - If the goal is shallow and to the right in the tree
- When will DFS outperform BFS? - When there are many deep goals in the tree, so you need to go deep anyway (memory limitations – DFS outperforms BFS)
- **Space** versus **Time** or **Quality** of Solution (trade-off)

Iterative Deepening

- **Idea:** get DFS's space advantage with BFS's time / shallow-solution advantages

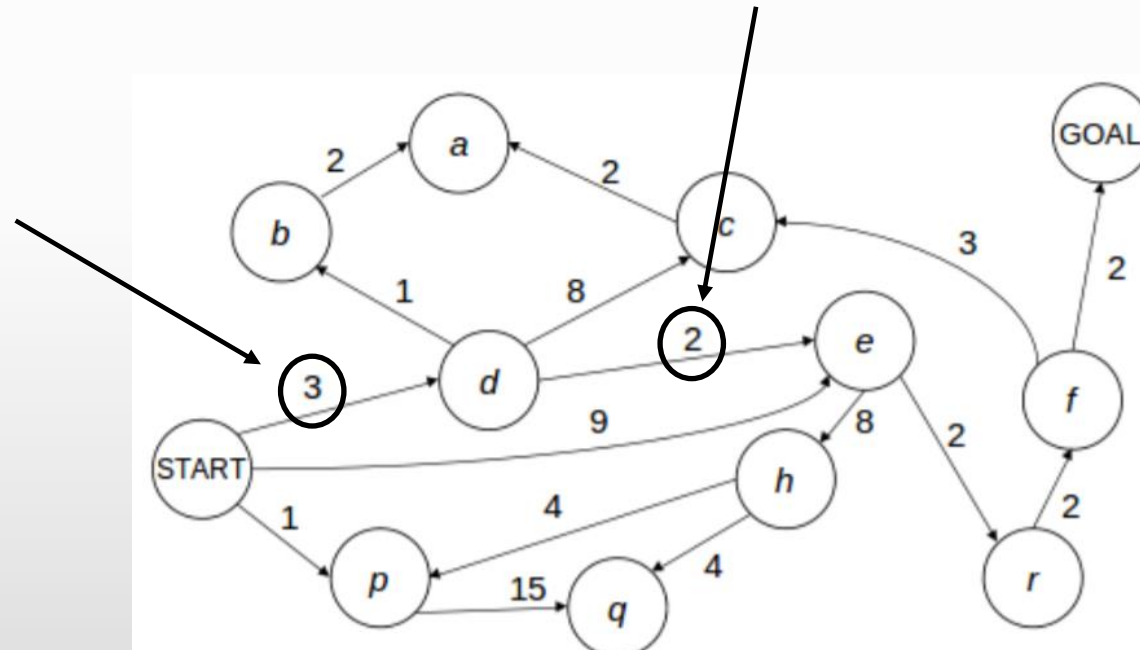


- Run a DFS (memory efficient) with depth limit 1
 - If no solution...
- Run a DFS with depth limit 2
 - If no solution ...
- Run a DFS with depth limit 3

Isn't that wastefully redundant? - Generally, most work happens in the lowest level searched, so **not so bad!** – Reason: the last layer is so much bigger than the previous layers

Cost-Sensitive Search

- BFS finds the shortest path in terms of number of actions (assuming all actions cost 1)
- It does not find the least-cost path. We will now cover a similar algorithm which does find **the least-cost path**

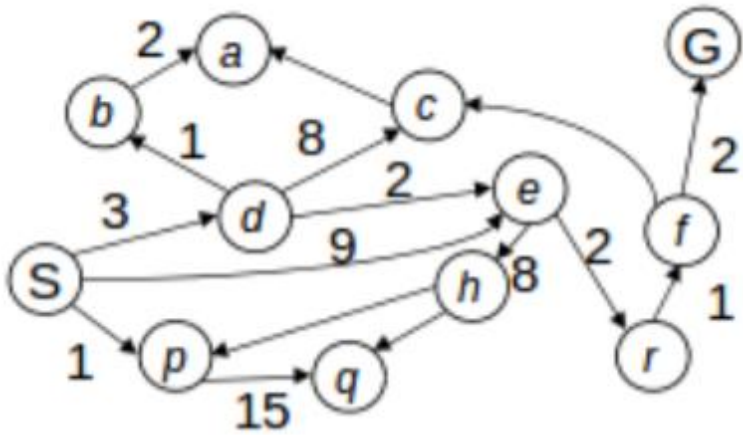


- **How?**

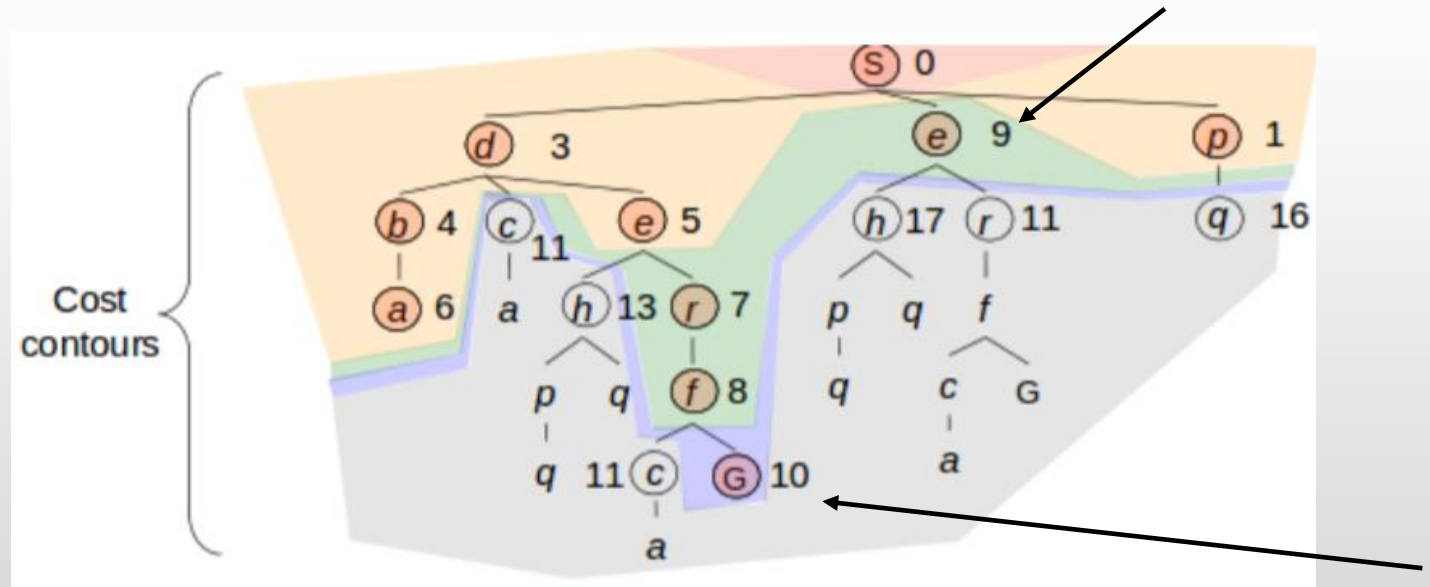
Uniform Cost Search



Uniform Cost Search (UCS)

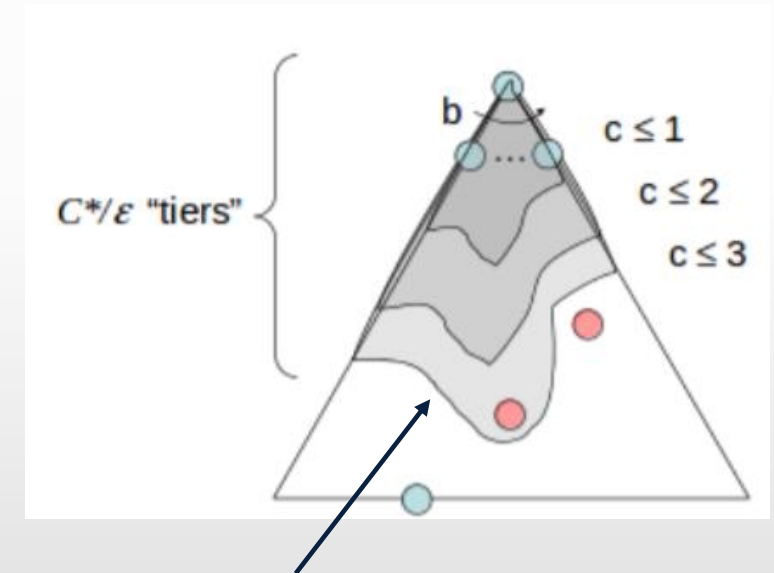


- **Strategy:** expand a **cheapest node first**
- **Fringe** is a **priority queue** (priority: lowest cumulative cost)



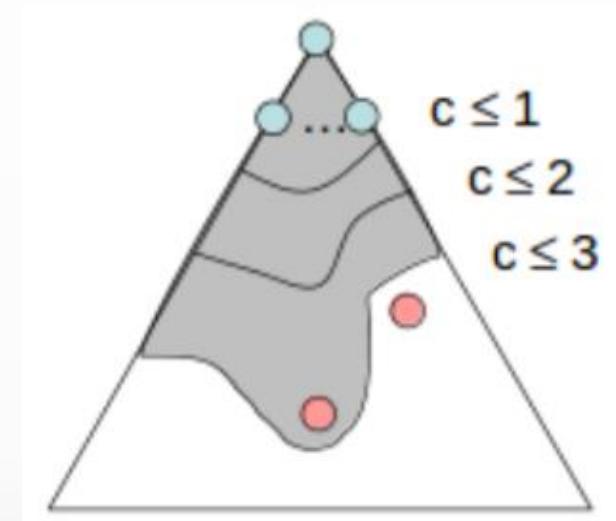
Uniform Cost Search - Properties

- What nodes does UCS expand?
 - If the optimal solution is a cost C^* , and each individual step costs us at least ϵ , then we might have expanded plans (longest path) that take C^*/ϵ (effective depth) actions to get there
 - All nodes cheaper than solution!
 - Solution cost C^* and arc cost $\geq \epsilon$: depth $O(C^*/\epsilon)$
- Time: $O(b^{C^*/\epsilon})$
- How much space does the fringe take?
- Last Tier Space: $O(b^{C^*/\epsilon})$
- Is it complete? Finite solution cost/positive arc weights? Then yes!
- Is it optimal? Yes (whenever we expand the goal state, everything on the fringe has a higher cost)



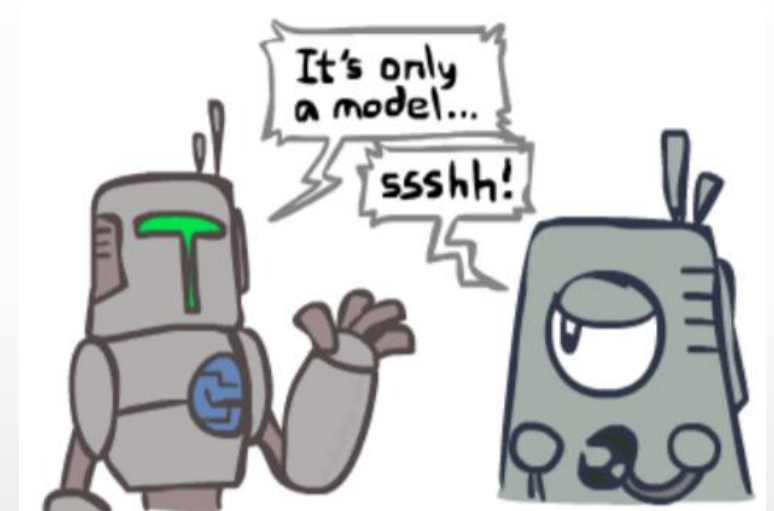
Uniform Cost Search – Issues

- Remember: UCS explores increasing cost contours
- The **good**: UCS is complete and optimal!
- The **bad**: Goes in every “direction”
- No information about goal location
- We’ll fix that soon!



Search and Models

- Search operates over models (state spaces) of the world
- The agent doesn't actually try all the plans out in the real world!
- Planning is all “in simulation”
- Your search is only as good as your models. . .



What you just learned and what's next?

- Problem-Solving Agents – **Planning Agents**
- **Search problems - Searching for Solutions**
- **Uninformed Search Methods**
- Next week - **Informed Search**
 - Heuristics
 - Greedy Search
 - A* Search
 - Graph Search

Questions?

A.DiStefano@tees.ac.uk